

WAR PREDICTION & CONFLICT SIMULATOR

— AI AGENT CONTEXT

Project Type: Winning Class Project | ML + Geopolitical Analysis + Interactive Simulation

Stack: Python · React · Streamlit · PyTorch/XGBoost · Three.js · Deck.gl

Audience: CS / Political Science students, professors, research evaluators

PROJECT OVERVIEW

This project is a **dual-purpose geopolitical intelligence system**:

1. **War Prediction Engine** — Uses historical conflict data, socioeconomic indicators, political instability indices, military strength metrics, and NLP-driven news sentiment to predict the probability of armed conflict between or within nations.
2. **War Outcome Simulator** — Given a simulated conflict scenario (two or more nations, resource parameters, alliance configurations), simulates probable outcomes using agent-based modeling and ML-powered casualty/duration forecasting.

The system is presented via two interfaces:

- **React Dashboard** — A stunning 3D geopolitical globe with real-time risk overlays, country drill-downs, and simulation controls (for demos and presentations)
 - **Streamlit App** — A data-heavy analytical interface for deeper exploration, model inspection, SHAP explanations, and running custom simulations (for judges/evaluators)
-

FEATURES LIST

Core Prediction Features

Feature	Description
Conflict Probability Score	0–100 score per country/dyad pair for next 1, 5, 10 years

Feature	Description
Risk Factor Breakdown	SHAP-based explanation of what's driving the risk score
Hot Zone Detection	Clustering countries by conflict risk into geo-spatial hot zones
Alliance Web Visualization	Graph showing military alliances, treaties, and dependencies
Temporal Trend Analysis	Risk score over time per region (line charts + heatmaps)
Conflict Type Classification	Civil war vs interstate vs proxy conflict classifier
News Sentiment Feed	Real-time NLP pipeline analyzing geopolitical news for tension signals

Simulation Features

Feature	Description
Custom Scenario Builder	Select 2+ nations, set military strength, economy, alliances
Agent-Based War Simulation	Step-by-step simulation of conflict escalation with stochastic agents
Outcome Forecasting	Duration, casualty estimate, economic damage, winner probability
Alliance Cascade Simulation	See if a conflict triggers a chain reaction via alliances (WWI model)
Nuclear Deterrence Toggle	Toggle nuclear capabilities and observe deterrence effects
Sanction Impact Modeling	Add economic sanctions and simulate pressure over time
Ceasefire Probability	Time-based probability of ceasefire or peace negotiation

Analytical Features

Feature	Description
Country Profile Cards	Military power index, GDP, stability score, past conflict history
Historical Conflict Explorer	Browse and filter 200 years of conflict data interactively
Model Comparison Panel	Compare XGBoost vs LSTM vs Logistic Regression predictions
Explainability Dashboard	SHAP waterfall charts, feature importance, decision trees
What-If Sliders	Adjust GDP, military spending, corruption index — watch score change
Export Reports	PDF/CSV export of scenario analysis and predictions

 TECH STACK

Backend (Python)

```
backend/

├── data/           # Raw + processed datasets

├── models/

|   ├── conflict_classifier.py    # XGBoost conflict type classifier
|   ├── probability_model.py      # Gradient Boosted Trees + LSTM ensemble
|   ├── outcome_forecaster.py     # Duration & casualty regression models
|   └── sentiment_pipeline.py     # HuggingFace FinBERT / geopolitical NLP

└── simulation/
```

```
| |─ agent_based_model.py    # Mesa framework: nation agents
| |─ alliance_cascade.py    # Graph-based alliance spread simulation
| └─ scenario_engine.py    # Orchestrates multi-agent war simulation
└─ api/
    | |─ main.py            # FastAPI app
    | |─ routes/
    | |   |─ predict.py
    | |   |─ simulate.py
    | |   └─ countries.py
    | └─ schemas.py
    └─ utils/
        | |─ feature_engineering.py
        | |─ shap_explainer.py
        | └─ data_loader.py
        └─ streamlit_app/
            | |─ app.py      # Main Streamlit entry
            | └─ pages/
                | |─ 01_prediction.py
                | |─ 02_simulator.py
                | |─ 03_explorer.py
                | └─ 04_model_insight.py
```

Layer	Technology	Purpose
ML Models	XGBoost, scikit-learn, PyTorch	Conflict prediction, classification
Deep Learning	LSTM via PyTorch	Temporal conflict sequence modeling
NLP	HuggingFace Transformers, NLTK	News sentiment, geopolitical signal extraction
Agent Simulation	Mesa (Python ABM framework)	Nation-agent war simulation
Graph Analysis	NetworkX	Alliance web, cascade modeling
Explainability	SHAP, LIME	Feature attribution, model transparency
API	FastAPI + uvicorn	REST endpoints for React frontend
Data Processing	Pandas, NumPy, GeoPandas	Feature engineering, geo operations
Visualization (py)	Plotly, Folium, Matplotlib	Charts within Streamlit
Streamlit	Streamlit + streamlit-extras	Analytical UI

Frontend (React)

frontend/

├─ src/

| └─ components/

| | └─ Globe/ # Three.js 3D globe with country meshes

| | | └─ GlobeScene.jsx # Main Three.js canvas

| | | └─ CountryMesh.jsx # Heat-colored country overlays

- | | | └─ ArcLines.jsx # Conflict arcs between nations
- | | └─ Dashboard/
- | | | └─ RiskPanel.jsx # Risk score cards
- | | | └─ AllianceGraph.jsx # D3/Force graph of alliances
- | | | └─ TrendChart.jsx # Recharts temporal risk
- | | └─ Simulator/
- | | | └─ ScenarioBuilder.jsx # Nation picker + parameter sliders
- | | | └─ BattleField.jsx # Animated 3D battle scene (Three.js)
- | | | └─ OutcomePanel.jsx # Results display
- | | └─ Shared/
- | | └─ CountryCard.jsx
- | | └─ ShapChart.jsx
- | | └─ NewsTickerFeed.jsx
- | └─ hooks/
- | | └─ useGlobe.js
- | | └─ useSimulation.js
- | | └─ useCountryData.js
- | └─ store/ # Zustand global state
- | └─ api/ # Axios API client
- | └─ App.jsx

Layer	Technology	Purpose
Framework	React 18 + Vite	Fast, modular UI
3D Globe	Three.js + react-three-fiber	Interactive 3D Earth
Globe Data	globe.gl or react-globe.gl	Prebuilt globe with arcs & heatmap
3D Battlefield	Three.js + @react-three/drei	Simulation scene (terrain, units)
Graph Viz	D3.js (force-directed)	Alliance network graph
Charts	Recharts + Victory	Trend lines, bar charts
Maps	Deck.gl + Mapbox GL	2D geospatial overlays
Animations	Framer Motion	Page transitions, UI micro-interactions
State	Zustand	Lightweight global state
Styling	Tailwind CSS + custom CSS vars	Military-dark theme
Icons	Lucide React	Clean iconography
HTTP	Axios	FastAPI communication

 DATASETS

Primary Datasets (Free, Publicly Available)

Dataset	Source	What it Contains
UCDP/PRIO Armed Conflict Dataset	Uppsala University	Every armed conflict 1946–present, parties, type, deaths

Dataset	Source	What it Contains
Correlates of War (COW)	correlatesofwar.org	Interstate wars, alliances, trade, MIDs (Militarized Disputes)
Global Terrorism Database (GTD)	START Center	200K+ terrorist events worldwide
ACLED	acleddata.com	Real-time conflict & protest events (API available)
World Bank Open Data	worldbank.org	GDP, population, military spending, poverty, corruption
SIPRI Military Expenditure DB	sipri.org	Military budgets, arms transfers per country
Fragile States Index	Fund for Peace	Political instability scores per country
Polity V / V-Dem	polityproject.org	Democracy scores, regime type
GDELT Project	gdeltproject.org	Global news events, tone, actors (updated every 15 min)
Global Peace Index	visionofhumanity.org	Annual peace scores per country
Nuclear Threat Initiative Index	nti.org	Nuclear/bio/chem security scores
UN Comtrade	comtrade.un.org	Trade dependencies between nations

Feature Engineering from Datasets

Sample engineered features per country-dyad per year

features = {

 # Political

"polity_score_A": -10 to +10, # Democracy score

"regime_type_A": "autocracy|democracy|anocracy",

"political_instability_index": 0-100,

"govt_effectiveness_score": WB indicator,

Economic

"gdp_per_capita_A": float,

"gdp_growth_rate_A": float,

"trade_dependency_AB": bilateral_trade / gdp, # higher = less likely to fight

"economic_sanctions_active": bool,

Military

"military_expenditure_pct_gdp_A": float,

"armed_forces_size_A": int,

"nuclear_capable_A": bool,

"military_power_index_A": composite, # from SIPRI + COW

Conflict History

"years_since_last_conflict": int,

"past_conflict_count_10yr": int,

"ongoing_conflict": bool,

"rivalry_score_AB": dyadic_rivalry_index,

Social

"ethnic_fractionalization": Alesina index,

"religion_fractionalization": float,

"human_development_index": UNDP HDI,

Geopolitical

"contiguous_border": bool,

"shared_alliance_AB": bool,

"UN_affiliation_same": bool,

"distance_km": float, # capital-to-capital

NLP / News

"news_sentiment_30d": float, # -1 to +1 from GDELT/news NLP

"hostile_event_count_30d": int, # from ACLED

"diplomatic_crisis_flag": bool,

}



ML MODELS

1. Conflict Probability Model

- **Algorithm:** XGBoost Classifier + LSTM ensemble

- **Target:** Binary `conflict_onset` (0/1) in next N years
- **Input:** Country-dyad features (see above)
- **Output:** Probability score 0–100
- **Evaluation:** AUC-ROC, Brier Score, Precision-Recall

2. Conflict Type Classifier

- **Algorithm:** Multi-class XGBoost
- **Classes:** `civil_war`, `interstate`, `proxy`, `terrorism`, `border_skirmish`
- **Features:** Regime type, ethnic indices, external actor involvement

3. Outcome Forecaster

- **Algorithm:** Gradient Boosted Regressor
- **Outputs:** `duration_days`, `estimated_casualties`, `economic_damage_usd`
- **Uncertainty:** Quantile regression for confidence intervals

4. News Sentiment NLP Pipeline

- **Model:** `ProsusAI/finbert` fine-tuned on geopolitical news OR `dslim/bert-base-NER`
- **Input:** GDELT + RSS news headlines
- **Output:** Sentiment score per country/dyad (rolling 7/30 day)

5. Agent-Based Simulation (Mesa)

```
class NationAgent(mesa.Agent):
```

```
    def __init__(self, country_id, military, economy, resolve, alliances):
```

```
        self.military_strength = military    # 0-100
```

```
        self.economic_health = economy      # 0-100
```

```
        self.resolve = resolve              # willingness to fight
```

```
        self.alliances = alliances          # list of allied agent ids
```

```
        self.status = "peace"              # peace | war | ceasefire | collapsed
```

```
    def step(self):
```

Check threat perception

Decide: escalate, de-escalate, invoke alliance, sue for peace

Update military strength (attrition), economy (war cost)

Probabilistic events: coup, ceasefire, nuclear use

UI DESIGN RECOMMENDATIONS

React Frontend — "WAR ROOM" Aesthetic

Theme: Military Operations Center — dark, high-contrast, data-dense, authoritative

Color Palette:

```
--bg-primary:    #080c10; /* Near-black navy */
--bg-surface:     #0d1117; /* Dark card surface */
--bg-elevated:    #161b22; /* Elevated panel */
--accent-red:     #ff3b3b; /* Danger / high risk */
--accent-amber:   #f59e0b; /* Warning / medium risk */
--accent-green:   #10b981; /* Safe / low risk */
--accent-blue:    #3b82f6; /* Neutral / info */
--accent-cyan:    #06b6d4; /* Interactive / highlight */
--text-primary:   #e6edf3;
--text-muted:     #8b949e;
--border:         #21262d;
--glow-red:       0 0 20px rgba(255, 59, 59, 0.4);
--glow-cyan:      0 0 20px rgba(6, 182, 212, 0.4);
```

Typography:

- Display / Headings: Orbitron (geometric, military-tech feel)
- Body: JetBrains Mono or IBM Plex Mono (data readout feel)
- Labels: Space Grotesk (clean, modern)

3D Components:

1. Globe (Main Hero) — react-globe.gl

- Countries colored by conflict risk (green → amber → red)
- Animated arcs between nations in active conflict
- Pulse animations on high-risk countries
- Click to drill into country details panel
- Atmosphere shader glow effect

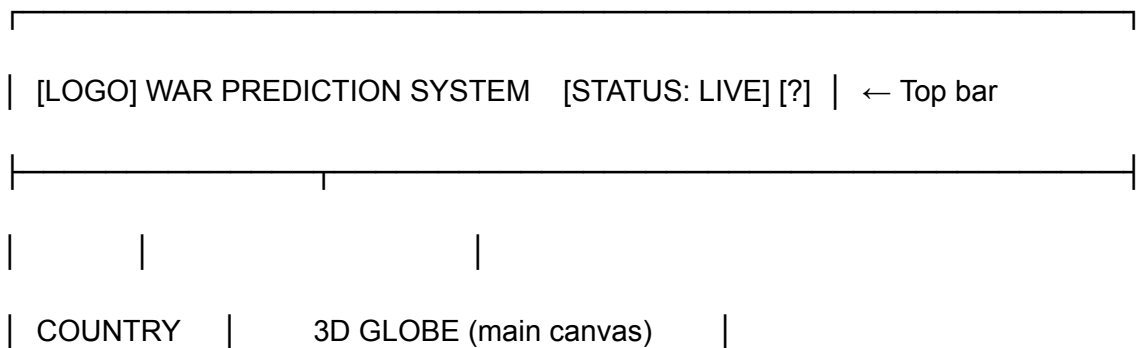
2. Battlefield Simulator Scene — react-three-fiber

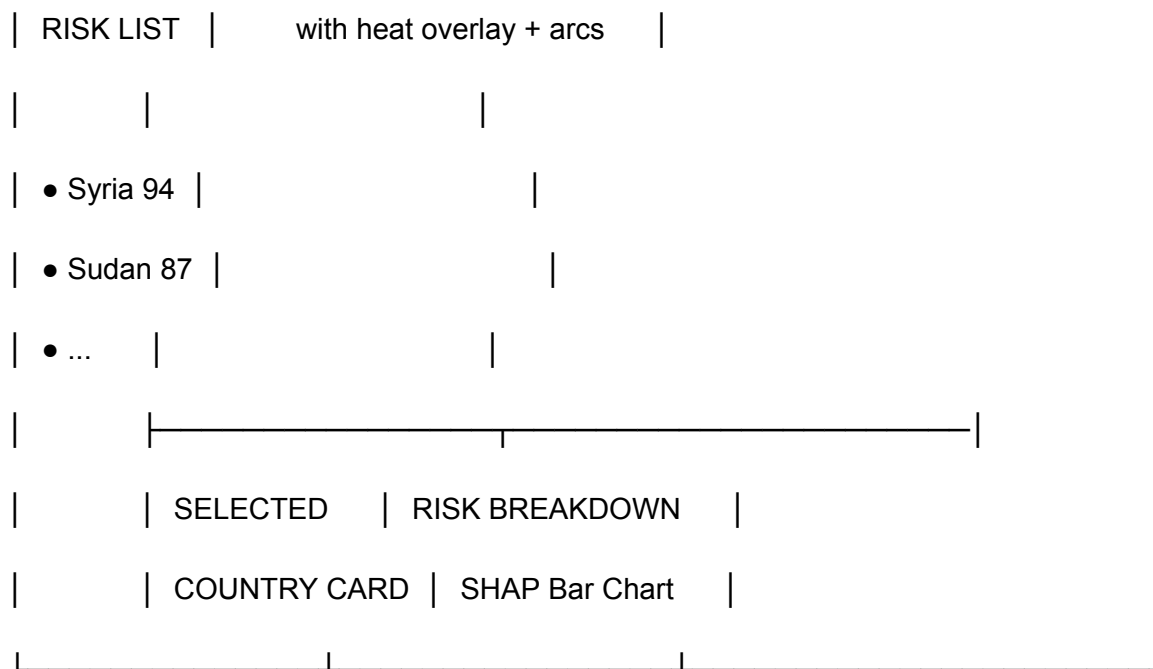
- Low-poly terrain (plane geometry + displacement map)
- Animated unit counters (red vs blue cubes/sprites)
- Particle explosions on conflict events
- Camera orbit controls
- Fog of war effect

3. Alliance Force Graph — D3 force simulation

- 3D positioned nodes (countries as spheres)
- Edge thickness = alliance strength
- Color: NATO blue / Russia red / China gold / neutral gray
- Animated links during cascade simulation

Page Layout:





[TABS]: Overview | Simulator | Explorer | Intelligence | Model Insights

Streamlit App — "Analyst Terminal" Aesthetic

Theme: Intelligence analyst workstation — functional but polished

Streamlit Config:

```
# .streamlit/config.toml
```

```
[theme]
```

```
primaryColor = "#3b82f6"
```

```
backgroundColor = "#080c10"
```

```
secondaryBackgroundColor = "#0d1117"
```

```
textColor = "#e6edf3"
```

```
font = "monospace"
```

Streamlit Pages:

1. 🌐 **Prediction Dashboard** — Country selector, risk score display, SHAP waterfall

2.  **War Simulator** — Scenario builder form, simulation runner, outcome charts
3.  **Conflict Explorer** — Filterable historical conflict table, timeline charts
4.  **Model Insights** — Feature importance, confusion matrix, AUC-ROC curve
5.  **Intelligence Feed** — Live news sentiment, GDELT event feed, NLP output

Key Streamlit Components:

```
import streamlit as st
```

```
import plotly.graph_objects as go
```

```
from streamlit_extras.metric_cards import style_metric_cards
```

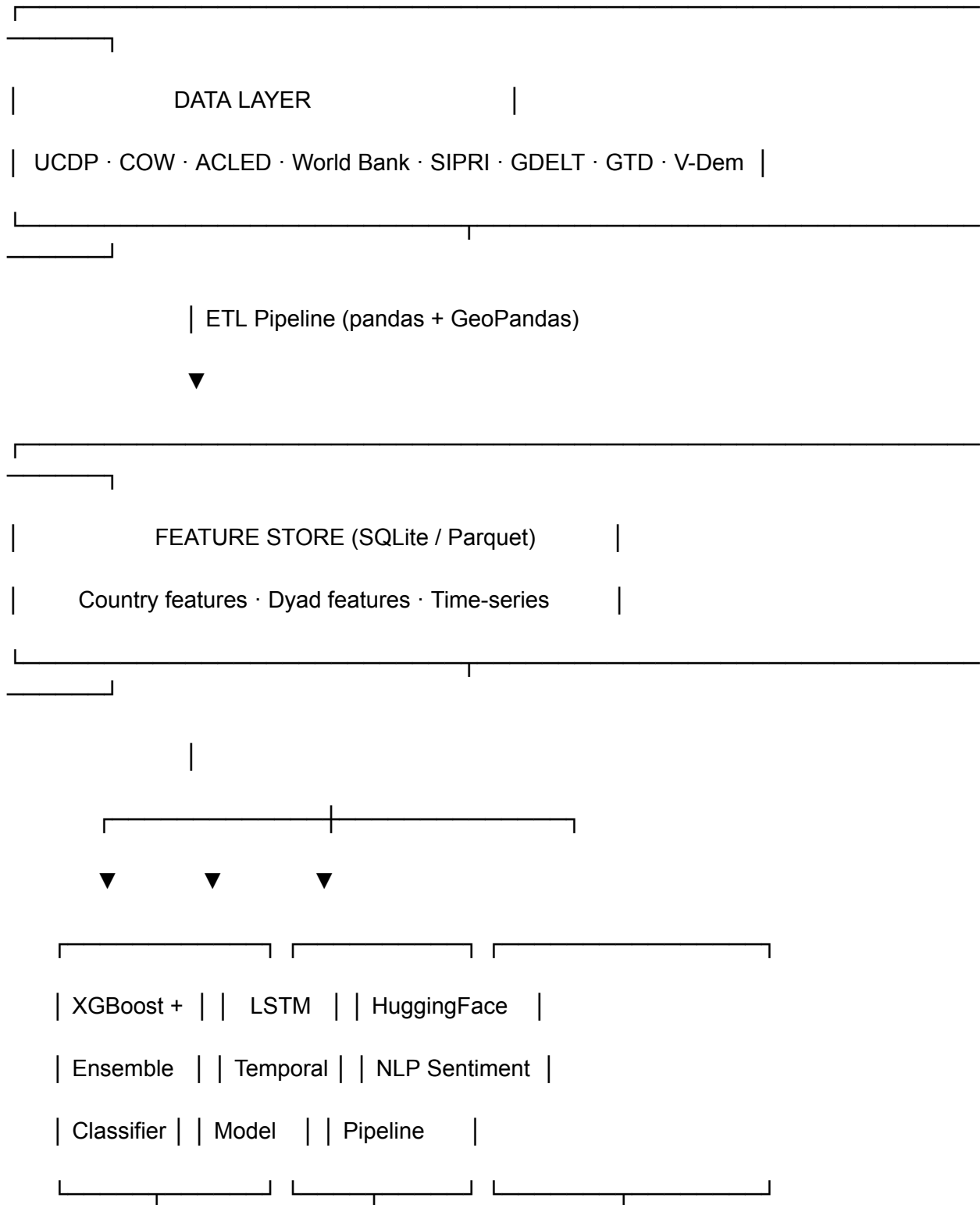
```
import pydeck as pdk # 3D map in streamlit
```

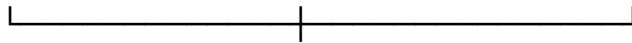
```
# 3D Map in Streamlit using PyDeck
```

```
layer = pdk.Layer(  
    "HexagonLayer",  
    data=conflict_data,  
    get_position=["longitude", "latitude"],  
    radius=200000,  
    elevation_scale=4,  
    elevation_range=[0, 1000000],  
    pickable=True,  
    extruded=True,  
    get_fill_color="[risk * 255, (1-risk) * 100, 50, 200]",  
)  
  
st.pydeck_chart(pdk.Deck(layers=[layer], initial_view_state=view_state))
```



ARCHITECTURE DIAGRAM



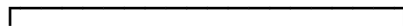
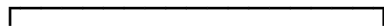
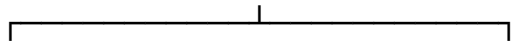
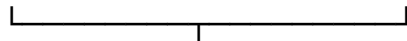


| FastAPI REST |

| /predict |

| /simulate |

| /countries |



| React Frontend | | Streamlit App |

| (War Room UI) | | (Analyst Tool) |

| Three.js Globe | | PyDeck 3D Map |

| D3 Alliance Web | | Plotly Charts |

| Framer Motion | | SHAP Plots |



FULL PROJECT STRUCTURE

war-prediction-system/

|

```
|— README.md

|— CONTEXT.md          ← (this file)

|— requirements.txt

|— docker-compose.yml

|

|— data/

| |— raw/              # Downloaded datasets

| |— processed/        # Cleaned, merged features

| |— feature_store.parquet

|

|— notebooks/

| |— 01_data_exploration.ipynb

| |— 02_feature_engineering.ipynb

| |— 03_model_training.ipynb

| |— 04_shap_analysis.ipynb

| |— 05_simulation_testing.ipynb

|

|— backend/

| |— models/

| | |— conflict_classifier.py

| | |— probability_model.py
```

```
| | └─ outcome_forecaster.py
| | └─ sentiment_pipeline.py
| └─ simulation/
| | └─ agent_based_model.py    # Mesa simulation
| | └─ alliance_cascade.py
| | └─ scenario_engine.py
| └─ api/
| | └─ main.py                # FastAPI
| | └─ routes/
| | └─ schemas.py
| └─ utils/
|   └─ data_loader.py
|   └─ feature_engineering.py
|   └─ shap_explainer.py
|
└─ streamlit_app/
  └─ app.py
  └─ pages/
    └─ 01_prediction.py
    └─ 02_simulator.py
    └─ 03_explorer.py
    └─ 04_model_insights.py
```

```
| | └─ 05_intelligence_feed.py
|
| └─ .streamlit/
|
|   └─ config.toml
|
|
└─ frontend/                # React app
  | └─ package.json
  | └─ vite.config.js
  | └─ src/
  |   └─ App.jsx
  |   └─ components/
  |     | └─ Globe/
  |     | └─ Dashboard/
  |     | └─ Simulator/
  |     └─ Shared/
  |   └─ hooks/
  |   └─ store/
  |   └─ api/
  |
  └─ tests/
    └─ test_models.py
    └─ test_simulation.py
```

IMPLEMENTATION ROADMAP

Phase 1 — Data & Models (Week 1–2)

- ☐ Download and merge UCDP, COW, World Bank, SIPRI datasets
- ☐ Build feature engineering pipeline (country-year + dyad-year)
- ☐ Train XGBoost baseline conflict probability model
- ☐ Evaluate with AUC-ROC, tune hyperparameters
- ☐ Add LSTM for temporal conflict sequences
- ☐ Set up SHAP explainability

Phase 2 — Simulation Engine (Week 2–3)

- ☐ Build **NationAgent** with Mesa framework
- ☐ Implement alliance cascade logic with NetworkX
- ☐ Create scenario engine (conflict start → steps → outcome)
- ☐ Integrate outcome forecaster model into simulation

Phase 3 — NLP Pipeline (Week 3)

- ☐ Set up GDELT data fetcher (15-min updates)
- ☐ Sentiment scoring with HuggingFace model
- ☐ Integrate sentiment as rolling feature into prediction model

Phase 4 — Streamlit App (Week 3–4)

- ☐ Build all 5 pages with polished dark theme
- ☐ Integrate PyDeck 3D maps
- ☐ Add SHAP waterfall plots, model comparison
- ☐ What-If slider panel

Phase 5 — React Frontend (Week 4–5)

- ☐ Set up Vite + React project, Tailwind
- ☐ Build 3D globe with react-globe.gl
- ☐ Add country risk heatmap + conflict arcs
- ☐ Build simulator scene with react-three-fiber
- ☐ Connect to FastAPI backend

Phase 6 — Polish & Demo (Week 5–6)

- ☐ Dockerize everything
 - ☐ Write README with screenshots
 - ☐ Record demo video
 - ☐ Prepare presentation slides
-

WINNING CLASS PROJECT TIPS

1. **SHAP Explainability** — Judges love "why did the model say this?" Show SHAP waterfall charts prominently.
 2. **Historical Validation** — Backtest your model on known conflicts (2003 Iraq, 2022 Ukraine) and show it predicted them.
 3. **Live Demo** — Use GDELT or ACLED API for live data so the demo feels real.
 4. **Ethical Framing** — Frame it as a *conflict prevention* tool, not a warfare optimizer. Mention limitations clearly.
 5. **Uncertainty Quantification** — Show confidence intervals, not just point estimates.
 6. **The 3D Globe** — This is your wow factor. Judges will remember it.
 7. **Alliance Cascade** — WWI demo ("what if Serbia-Austria conflict triggers all of Europe?") is incredibly compelling.
 8. **Video Walkthrough** — Record a 3-minute demo video even if presenting live.
-

ETHICAL CONSIDERATIONS (Required Section)

- This system is designed for **academic research and conflict prevention awareness**
 - Predictions are **probabilistic, not deterministic** — model output $\neq$ ground truth
 - All data used is **publicly available academic/government data**
 - The simulation engine is a **simplified model** and should not be used for actual military planning
 - Model biases must be acknowledged: historical data reflects past power structures
 - Include a **bias audit** comparing prediction rates across Global North vs Global South
-

KEY DEPENDENCIES

requirements.txt

fastapi==0.104.0

uvicorn==0.24.0

pandas==2.1.0

numpy==1.24.0

scikit-learn==1.3.0

xgboost==2.0.0

torch==2.1.0

transformers==4.35.0

mesa==2.1.0

networkx==3.2.0

shap==0.43.0

geopandas==0.14.0

plotly==5.17.0

streamlit==1.28.0

pydeck==0.8.0

folium==0.15.0

requests==2.31.0

gdelt==0.1.15

acled-py==1.0.0

scipy==1.11.0

// package.json (frontend key deps)

{

```
"dependencies": {  
  
  "react": "^18.2.0",  
  
  "three": "^0.158.0",  
  
  "@react-three/fiber": "^8.15.0",  
  
  "@react-three/drei": "^9.88.0",  
  
  "globe.gl": "^2.26.0",  
  
  "d3": "^7.8.5",  
  
  "recharts": "^2.9.0",  
  
  "framer-motion": "^10.16.0",  
  
  "zustand": "^4.4.0",  
  
  "axios": "^1.6.0",  
  
  "tailwindcss": "^3.3.0",  
  
  "deck.gl": "^8.9.0"  
  
}  
  
}
```